Language Cadence II: A Hybrid Syllable Tokenizer and a Null Result for Periodic Positional Encoding

Navid Nabijou*

October 1, 2026

Abstract

A companion paper (Nabijou, 2026) tested whether tuning the period of a strictly periodic positional encoding per language improves GPT pretraining, and found a null result traced to a boundary effect – a purely periodic encoding makes a longer period indistinguishable from a longer usable context – compounded by a character-level tokenizer, under which a period has no interpretable linguistic meaning. This paper addresses both for English, first building and validating a hybrid syllable tokenizer (CMUdict counts, a spelling-based cut-point algorithm, and an out-of-vocabulary fallback) that we expect to be useful beyond this project wherever English text needs a well-behaved, linguistically grounded syllable-level unit, then adding the periodic component on top of a standard (Vaswani-style) base encoding, which removes the boundary effect directly by keeping absolute-position information available regardless of the period. Sweeping 8 periods with 3 paired seeds each, against a no-periodic baseline and a RoPE control (30 runs, English, syllable-tokenized TED-talk transcripts), we again find a negative result, but of a different character: every periodic configuration underperforms baseline, consistently in direction, yet no individual period is significant after correcting for multiple comparisons. An omnibus test shows the periods differ meaningfully as a group ($p < 0.0001$) even though no specific pairwise gap survives correction – a real effect not resolvable pointwise at this sample size. RoPE outperforms both by a wide margin, confirming the setup can detect a real positional effect. The training curves reveal a further anomaly: the periodic-vs-baseline gap is U-shaped over training, nearly closing mid-run before reopening late, unexplained by anything in the design.

1 Introduction

Nabijou (2026) set out to test the idea that different natural languages might have an intrinsic "cadence": a best-performing period for a periodic positional encoding. The experiment swept that period across five languages, and for each language read off whichever period minimized next-token pretraining loss. The result was a clean null, but a diagnosable one. For every language tested, loss decreased monotonically as the period grew, all the way up to the model's context length. The paper traced this to what it called the *boundary effect*. The periodic encoding was the model's *only* source of positional information, so a period close to the context length N stops repeating within any single training window. This is functionally equivalent to handing the model a longer usable context – an improvement that has nothing to do with periodicity as such. The same paper flagged a second, compounding concern, though it was not able to test it directly: its tokenizer operated at the character level. A character has no principled relationship to syllable- or

*Queen Mary University of London. nn333@cantab.ac.uk

clause-level linguistic structure, so a "period" measured in characters cannot be interpreted as a period in any unit that a genuine rhythmic effect would plausibly operate over.

This paper is a direct sequel, addressing both problems, but narrowed to a single, more tractable question. For English specifically: does adding a periodic positional-encoding component on top of a normal positional encoding improve training efficiency, as measured by test loss at a fixed training budget? (The companion paper instead compared a per-language optimum across five languages.) Two design changes follow directly from the companion paper's diagnosis. First, the periodic component is now added on top of a fixed, standard (Vaswani-style) base positional encoding. Full absolute-position information is therefore always available to the model, regardless of the chosen period, and this removes the mechanism behind the boundary effect at the architectural level. In the companion paper, by contrast, the periodic encoding was the model's only positional signal; here it is strictly additional. Second, we build a dedicated hybrid syllable tokenizer for English, so that a candidate period corresponds to a linguistically motivated unit: the syllable. The companion paper tokenized at the character level, which is an orthographic unit with no principled relationship to syllable structure.

The tokenizer is, in our view, as substantial a contribution of this project as the positional-encoding experiment itself. We therefore give it a correspondingly detailed treatment in Section 2, covering its design, its validation against CMUdict ground truth, and the vocabulary built from it. It was built to give the period sweep an interpretable unit, but nothing about its design is specific to that purpose. We expect it may be of independent interest for other English NLP work that wants syllable-level granularity, such as phonologically or prosodically motivated modeling, text-to-speech front ends, or readability and rhythm analysis.

The rest of the paper proceeds as follows. Section 3 describes the remaining experimental method: the positional-encoding design, the dataset, and the sweep. Section 4 reports the results, including a formal statistical analysis. That analysis addresses a question any reader of a sweep plot should ask: is the period-to-period structure visible in the data distinguishable from noise, or is it an artifact of how the plot happens to be scaled? Section 5 offers deliberately labeled speculation about what might be driving the negative result and its internal structure. Section 6 states the caveats and open questions plainly.

2 Tokenizer

Character-level tokenization treats every language identically, regardless of its structure. This is attractive for cross-linguistic comparison, but the companion paper identified it as a likely reason the periodic-encoding idea was never given a fair test. A period expressed in characters is not interpretable as a period in any unit that a rhythmic effect would plausibly operate over. This is because a fixed number of characters corresponds to a different amount of linguistic structure depending on orthographic depth: compare English "th" or German "sch," each one phoneme spelled with two characters, to a language with a shallower orthography. The natural unit to test the cadence hypothesis against is instead something closer to the syllable. (The mora is a finer-grained unit in the general prosodic hierarchy, and is the relevant rhythmic unit for mora-timed languages such as Japanese, but it plays no comparable role in English phonology. For an English-only tokenizer, the syllable is therefore the finest unit that is actually doing linguistic work.) Standard subword tokenizers, such as byte-pair encoding and its relatives, were ruled out for the same reason as the character-level choice. Their piece boundaries are chosen to minimize a compression objective, not to track any linguistic unit, so a period measured in BPE pieces would carry exactly the same interpretability problem that the companion paper diagnosed.

We therefore built a dedicated *hybrid syllable tokenizer* for English: a piece boundary placement that is anchored to a genuine linguistic count (dictionary syllable counts) wherever that ground truth is available, with a validated fallback where it is not.

2.1 Design

The tokenizer operates on lowercased text and produces a flat stream of pieces. Each word becomes one or more syllable pieces. The first piece of each word carries a leading marker _, so word boundaries are always recoverable from the piece stream alone. Punctuation characters and single digits are emitted as standalone pieces. Syllabifying a single word proceeds in two stages.

Syllable count. For a word present in the CMUdict pronunciation dictionary (CMU, 1998), the syllable count k is the number of vowel phones in its pronunciation. (CMUdict phones are stress-marked, so a phone's syllabicity can be read directly off a trailing digit.) Roughly 1.9% of CMUdict entries have more than one listed pronunciation, with disagreeing syllable counts between them; we resolve this by majority vote across the pronunciations, breaking any remaining tie in favor of the first-listed one. Some words are absent from CMUdict altogether: these are out-of-vocabulary (OOV) with respect to the dictionary, and account for about 1.6% of word tokens in our evaluation corpus. For an OOV word, k is instead estimated by a fallback procedure. The fallback takes a majority vote across three signals: a vowel-group heuristic (counting maximal runs of vowel letters), and the syllable counts implied by two independent automatic syllabifiers, `pyphen` (Silva, 2013) and NLTK's Sonority Sequencing Principle (SSP) tokenizer (Bird et al., 2009).

Cut placement. Given k, the word's $k-1$ internal cut points must be chosen from the $L-1$ inter-letter boundaries of an L-letter word. We do this with a dynamic program that scores each candidate set of boundaries. Four signals contribute to the score: agreement with `pyphen`'s own hyphenation points; agreement with the SSP splitter's boundaries; a bonus when the syllable immediately following a cut begins with a licensed English consonant cluster (a fixed allow-list of legal onsets, e.g. *bl-*, *str-*, *th-*); and small bonuses for a handful of common morphological suffix boundaries (*-ing*, *-tion*, *-ment*, and similar). Two classes of word are special-cased, rather than left to this general scoring. The first is contractions: a clitic (*n't*, *'s*, *'ll*, ...) is split off as its own final piece, so *didn't* becomes *did-n't*. The second is words ending in a consonant plus syllabic *-le* (*-ble*, *-cle*, *-dle*, ...); that ending is treated as a syllable nucleus in its own right, even though it contains no written vowel, so *enabling* becomes *en-a-bl-ing*.

This design was deliberately chosen over either automatic syllabifier alone. On a held-out evaluation sample (the Brown corpus; see Section 2.2), `pyphen` alone matches CMUdict's syllable *count* for only 79.3% of out-of-vocabulary-probe words, and NLTK's SSP splitter is weaker still. Each tool encodes a different, imperfect model of English syllabification: `pyphen` tends to under-split short words, while SSP tends to over-split silent-*e* words such as *time* or *have*. Anchoring the *count* to CMUdict wherever possible, and only falling back to a vote among the automatic tools where CMUdict has no entry, removes most of this disagreement by construction, for the 98.4% of word tokens that CMUdict does cover.

2.2 Validation

We evaluate the tokenizer on the Brown corpus (991,429 word tokens, 42,256 distinct word types), which is independent of both the training corpus used for the main experiment (Section 3.2) and of CMUdict itself. Table 1 summarizes the key statistics.

By construction, the pieces always concatenate back to the original word. The first row after dictionary coverage is therefore guaranteed rather than measured; we include it only as a sanity

Table 1: Hybrid syllable tokenizer, evaluated on the Brown corpus.

Quantity	Value
CMUdict coverage (word tokens)	98.4%
CMUdict coverage (distinct word types)	87.8%
Pieces rejoin the original word	100.0%
Words with a vowel letter in every piece	98.4%
Mean syllables per word	1.56
Mean characters per syllable piece	3.03
Fallback syllable-count accuracy (OOV probe)	83.0%
vs. `pyphen` alone	79.3%

check that the implementation has no silent bugs. (The next row, vowel-in-every-piece, is a genuinely measured quantity: piece counts summing to exactly k does not guarantee that every piece contains a vowel letter, which is why this row falls short of 100%.) The more informative numbers are coverage and fallback accuracy. Coverage measures how often the gold-standard count is even available; fallback accuracy measures how good our estimate is when it is not.

To measure fallback accuracy, we deliberately treated a subset of CMUdict-covered words as if they were out-of-vocabulary, so that we could compare the fallback's estimated syllable count against the true, dictionary-derived count. On this probe, the fallback – a majority vote across three independent estimators – outperforms the best single automatic syllabifier (`pyphen`) by 3.7 points, even though `pyphen` is itself one of the fallback's three voters. This confirms that combining signals is worth the added complexity.

The weakest statistic we measured is agreement with `pyphen`'s specific *cut positions*, as opposed to syllable *counts*, on the subset of words where both tools agree on the count. Only 60.1% of such words receive identical cut placement from the two methods. We read this as expected rather than concerning. When the count is already fixed, the dynamic program's additional scoring terms (the legal-onset and suffix bonuses, which `pyphen` does not use) are free to move the cut to a linguistically better-justified position than `pyphen`'s own heuristic. We also have no independent ground truth for cut *placement*, as opposed to count, with which to adjudicate between the two methods, so we report the discrepancy for transparency rather than treating either tool as gold-standard here.

2.3 Vocabulary

We built a piece vocabulary directly from the training split of the dataset described in Section 3.2 (6,398,658 training pieces, 19,286 distinct piece types), keeping pieces by frequency rank. Out-of-vocabulary pieces fall back to a character-level encoding, so no input is ever unrepresentable. Figure 1 shows coverage as a function of vocabulary size, on both the training split itself and the held-out validation split. We fixed the vocabulary at 8,000 pieces, which covers 99.5% of training tokens and 99.2% of held-out validation tokens. Past this point, the coverage curve has visibly flattened. Vocabulary sizes above 16,000 do approach 100% coverage, but at roughly double the parameter cost in the model's embedding and output layers, so the training-time cost of a larger output layer is not justified by the small additional coverage it would buy.

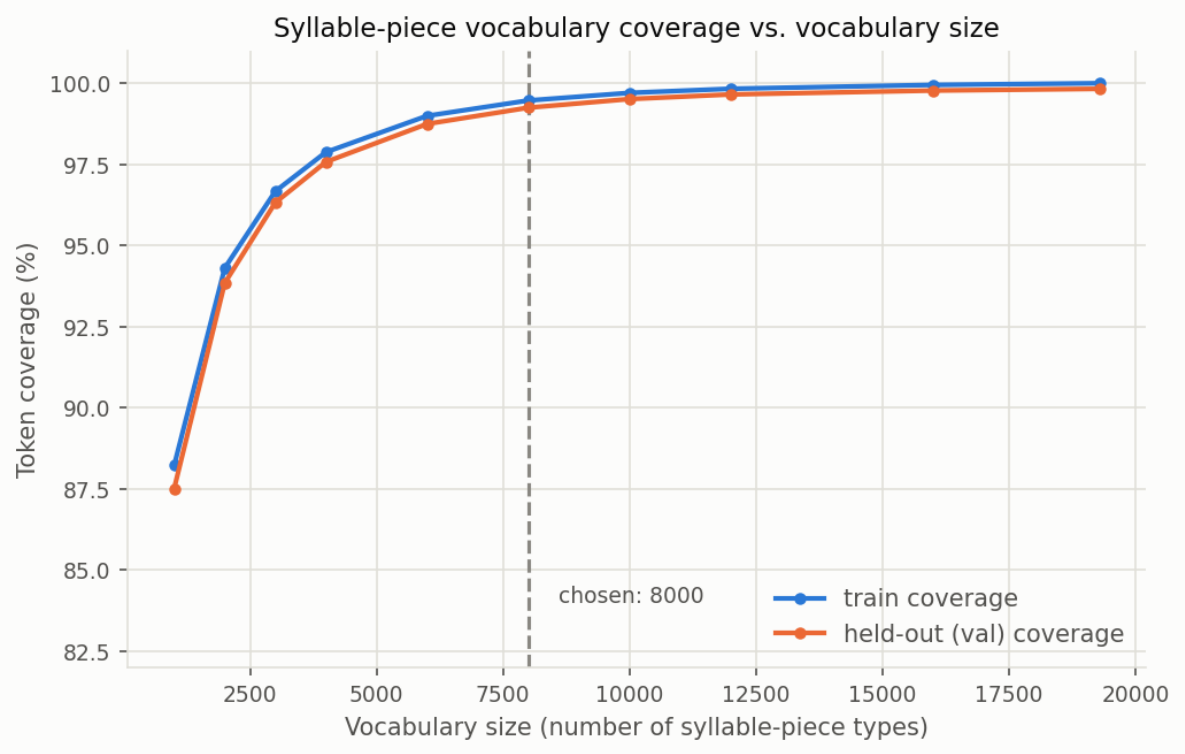


Figure 1: Syllable-piece vocabulary coverage as a function of vocabulary size, measured on the training split (used to build the vocabulary) and the held-out validation split. The chosen size, 8,000 pieces, is marked.

2.4 Known weaknesses

Three limitations of the tokenizer are worth stating plainly, since they bear directly on how the main experiment's results should be interpreted; Section 6 returns to each. First, CMUdict reflects General American pronunciation, while TED speakers are drawn from many English varieties. The dictionary's syllable counts are therefore not uniformly accurate ground truth for every speaker's actual pronunciation, even where a word is covered. Second, the tokenizer counts *canonical* dictionary syllables, not syllables as they were actually realized in any given utterance. Connected speech routinely elides or merges syllables in ways no dictionary look-up can capture. Third, cut-*placement*, as opposed to count, is validated only by agreement with other heuristic tools, and not against any independent ground truth, as noted above. None of these three limitations affects the syllable *counts* that ultimately determine the piece stream's granularity, which is the quantity most relevant to testing a periodic-encoding hypothesis. They do mean, however, that the tokenizer should be read as a substantially more linguistically grounded unit than characters or BPE pieces, rather than as a perfect phonetic transcription. These limitations are modest relative to what character- or BPE-level tokenization offers in the way of syllable awareness, which is essentially none. The tokenizer should therefore still be a useful building block for other syllable-aware work on English, independent of the periodic-encoding question it was built to serve here.

3 Method

3.1 Positional encoding

As in the companion paper, each position $n \in \{0, \ldots, N-1\}$ is encoded by a D-dimensional vector, built from $H = D/2$ sine/cosine row-pairs. The periodic component is constructed so that it has an exact period P. Row-pair j (for $j = 1, \ldots, H$) is assigned the folded harmonic

$$k_j = \min\big(j \bmod P,\ P - (j \bmod P)\big),$$

This avoids wasting embedding capacity on harmonics that would otherwise alias, folding back onto an earlier one (up to sign) once j exceeds $P/2$; see Nabijou (2026) for the derivation. With this assignment, the periodic component $\mathbf{\Pi}_{\text{per}}$ has entries

$$\mathbf{\Pi}_{\text{per}}[2j-2, n] = \sin\left(\frac{2\pi k_j n}{P}\right), \qquad \mathbf{\Pi}_{\text{per}}[2j-1, n] = \cos\left(\frac{2\pi k_j n}{P}\right).$$

The crucial difference from the companion paper is what this periodic component is combined with. There, $\mathbf{\Pi}_{\text{per}}$ was the model's *entire* positional signal. This is precisely what allowed a long period to masquerade as extra usable context (Section 1). Here, we first compute a standard, fixed, non-periodic base encoding $\mathbf{\Pi}_{\text{base}}$: the geometric frequency spread of Vaswani et al. (2017), with entries $\mathbf{\Pi}_{\text{base}}[2i, n] = \sin(n/10000^{2i/D})$ and $\mathbf{\Pi}_{\text{base}}[2i+1, n] = \cos(n/10000^{2i/D})$. We then form the combined encoding

$$\mathbf{\Pi}(n) = \mathbf{\Pi}_{\text{base}}(n) + \mathbf{\Pi}_{\text{per}}(n; P).$$

Because $\mathbf{\Pi}_{\text{base}}$ is present regardless of P, the model always has access to full absolute-position information. The periodic component can only ever add to that; it can never substitute for it. This directly removes the mechanism behind the boundary effect: a large P no longer corresponds to "more usable context," since the base encoding already supplies a usable context signal of its own, constant across the sweep. A *no-periodic-component* configuration, $\mathbf{\Pi}(n) = \mathbf{\Pi}_{\text{base}}(n)$ alone, serves as the baseline against which every periodic configuration is compared (Section 3.4).

$\mathbf{\Pi}(n)$ is injected by concatenating it with the token embedding, then passing the result through a learned linear projection back to D dimensions: $\mathbf{x} = \text{Linear}(\text{concat}(\mathbf{e}_{\text{tok}}, \mathbf{\Pi}))$. We chose concatenation over additive injection following a calibration pilot in the companion paper's own project. That pilot found that additive injection has a narrow, saturating useful amplitude range in a pre-norm transformer. Every block here opens with LayerNorm, and $\text{LayerNorm}(c\mathbf{v}) = \text{LayerNorm}(\mathbf{v})$ for any positive scalar c. Past a small amplitude threshold, then, the positional signal's relative contribution is clamped, regardless of how large it is made. Concatenation sidesteps this fragility, at the cost of one extra learned projection layer and with no amplitude hyperparameter to tune. We use it for every configuration in the sweep below, including the baseline, so the injection method is held fixed and cannot confound the comparison.

As a stronger positional-encoding control, we also train a *rotary position embedding* (RoPE) configuration (Su et al., 2021). RoPE conveys position by rotating the query and key vectors inside each attention head, by an angle that is a function of relative position, before the attention dot product is computed. This is a structurally different mechanism, acting directly on what attention can represent. We include it as a positive control: a configuration already well established as effective, used to confirm that the overall pipeline (architecture, data, training protocol) is capable of detecting a real positional-encoding effect when one is present. We do not use it to test the periodicity hypothesis itself.

3.2 Dataset

Training data is the English source side of the WIT3/IWSLT TED-talk transcript collection (Cettolo et al., 2012), 2016 release. This gives 2,085 talks spanning three decades, or 4.3M words before preprocessing. We processed the talks in three ways: stripping parenthetical audience and audio cues such as "(Laughter)" and "(Applause)" (14,631 occurrences across the corpus); dropping the 11 talks that remained under 100 words after cleaning, which were near-wordless music or dance performances; and otherwise keeping casing, punctuation, and digits as written. The corpus was split by *speaker*, greedily balanced to a 90/5/5 word-count split with zero speaker overlap verified between splits. A talk-level split would instead have leaked a speaker's idiosyncratic vocabulary and style across the train/test boundary, since 221 of the corpus's 1,767 distinct speakers give more than one talk. The resulting splits are 1,667 talks, 3.89M words, and 1,363 speakers for training; 202 talks, 216k words, and 197 speakers for validation; and 205 talks, 216k words, and 198 speakers for test.

3.3 Architecture

The architecture is held fixed across every configuration in the sweep. It has embedding dimension $D = 128$, $H = 64$ positional-encoding row-pairs, 4 transformer layers, and 4 attention heads with head dimension 32, a context length of $N = 1{,}024$, and the 8,000-piece syllable vocabulary of Section 2. This comes to approximately 2.87M parameters in total. It is a deliberately small, laptop-scale model; Section 6 returns to what that implies for the results.

3.4 Sweep design and protocol

The period is swept over

$$P \in \{2, 3, 6, 9, 16, 45, 128, 1024\},$$

This grid is reallocated toward the low end relative to the companion paper's more evenly log-spaced grid, on linguistic grounds. These values span roughly the scale of a syllable, a short word, and a phrase, which is the range at which a genuine rhythmic effect operating on syllable-tokenized text would plausibly appear. $P = 128$ is retained as the companion paper's geometric-distinguishability optimum for $H = 32$, that paper's row-pair count (see its Section 2.1). We include it here as a direct cross-paper comparison point, even though $H = 64$ in this paper's own architecture shifts where that optimum would actually fall. $P = 1{,}024 = N$ is retained specifically as the boundary-effect check described in Section 3.1. With a base encoding now always present, this is the direct test of whether the companion paper's monotonic-to-the-boundary pattern persists once the mechanism behind it has been architecturally removed. $P = 45$ fills the resulting gap between the low cluster and $P = 128$.

Each of the 8 periodic configurations, plus the no-periodic baseline and the RoPE control, was trained with 3 random seeds. Critically, the *same* three seeds – identical parameter initialization and identical training-data shuffle order – are reused across all 10 configurations. This makes every pairwise comparison between two configurations a *paired* one (seed-matched). A paired comparison removes seed-to-seed noise common to both arms of any given comparison, and substantially increases statistical power at a fixed, small number of seeds (Section 4.2). In total, this gives $8 \times 3 + 3 + 3 = 30$ training runs.

Each run trains for 3,000 steps, at batch size 16 and a single fixed learning rate. This corresponds to roughly 7–8 passes over the 3.89M-word training split. Validation loss on a fixed evaluation subset is tracked every 100 steps throughout training (Section 4.3). We take the checkpoint with the lowest

such validation loss over the whole run, evaluate it on the full held-out test set, and report that as the run's result. This guards against the overfitting risk that a 7–8-pass budget invites. For the headline comparison between periods (Section 4.1), the single best-performing period is likewise selected using validation loss alone, so no part of the headline comparison is chosen by looking at the test set.

4 Experiments

All 30 runs completed without incident. No run diverged, checked against a hard non-finite-loss halt, or triggered a soft loss-spike warning, and every run reached its full 3,000-step budget. We report results in three parts: the headline test-loss comparison (Section 4.1); a formal statistical analysis of that comparison (Section 4.2); and an analysis of how loss evolves *during* training, rather than only where it ends up (Section 4.3).

4.1 Headline comparison

Table 2 and Figure 2 report mean test loss (cross-entropy, nats per syllable-piece[1], at the best-validation-loss checkpoint) across the 3 seeds for each of the 10 configurations.

Table 2: Mean test loss by configuration (3 seeds each, mean ± standard deviation).

Configuration	Test loss
Baseline (no periodic component)	3.9411 ± 0.0165
Periodic, $P = 2$	3.9641 ± 0.0042
Periodic, $P = 3$	3.9573 ± 0.0029
Periodic, $P = 6$	3.9611 ± 0.0049
Periodic, $P = 9$	3.9625 ± 0.0065
Periodic, $P = 16$	3.9648 ± 0.0044
Periodic, $P = 45$	3.9814 ± 0.0042
Periodic, $P = 128$	3.9959 ± 0.0073
Periodic, $P = 1024$	3.9825 ± 0.0078
RoPE	3.6141 ± 0.0068

Every one of the 8 periodic configurations is worse than the baseline. The best, validation-selected, period is $P = 3$, which also happens to match the period selected by test loss. The two selection criteria agree, which rules out the specific concern that a best-period claim here would reflect test-set peeking. RoPE, meanwhile, is a large, unambiguous win over both: a 0.33-nat gap to baseline, roughly an order of magnitude larger than any periodic-vs-baseline gap observed. We treat the RoPE comparison purely as the positive control described in Section 3.1. Its advantage over additive and concatenative positional encodings is already well established in the literature (Su et al., 2021), so it is not itself a finding of this paper, and we do not analyze it further below.

The pattern across periods is not monotonic. The four smallest periods, $P \in \{2, 3, 6, 9, 16\}$, cluster closely together and perform best among the periodic configurations. $P = 45$, and especially $P = 128$, are the worst points on the grid. $P = 1{,}024 = N$ partially, but not fully, recovers. This

[1]This paper tokenizes at the syllable-piece level; the companion paper tokenized at the character level. Loss values here are therefore not directly comparable to that paper's reported nats-per-character figures – a syllable piece carries, on average, 3.03 characters' worth of information (Table 1), so the two loss scales measure different-sized prediction units.

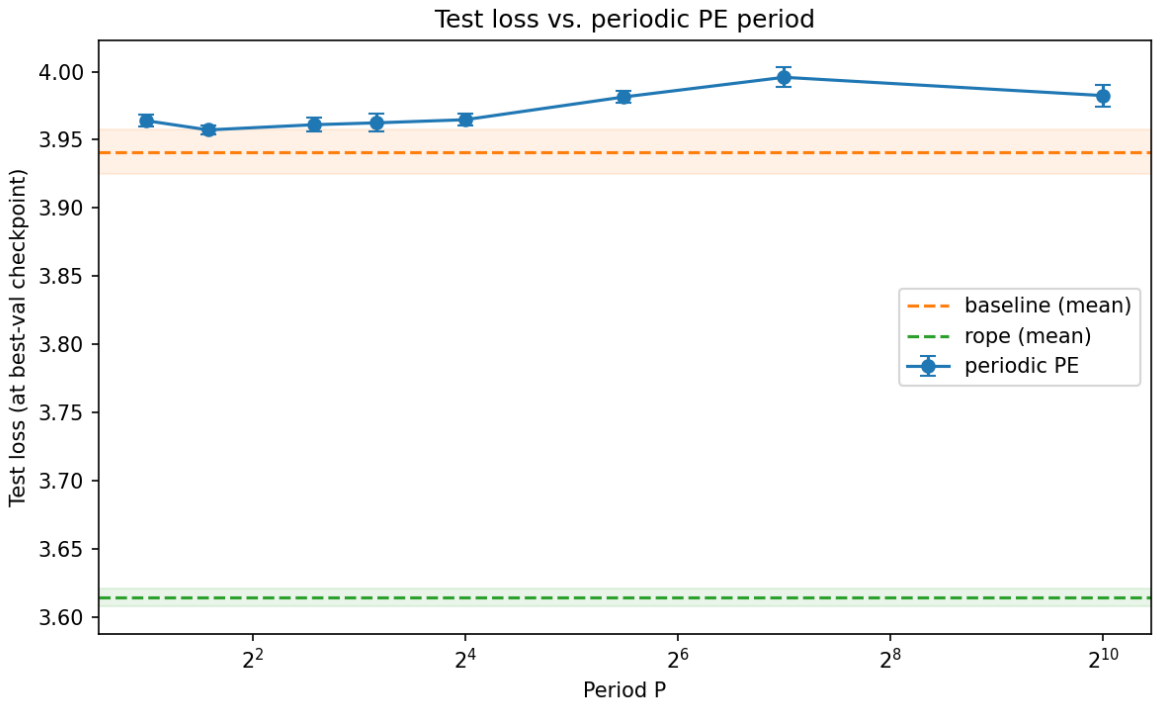


Figure 2: Test loss vs. period P (log-x), with the no-periodic baseline and RoPE shown as reference bands (mean $\pm$ 1 std.). Every periodic configuration lies above the baseline; RoPE lies well below both.

is qualitatively different from the companion paper's result, where loss decreased monotonically all the way to $P = N$ in every language, with no exception. Here, $P = N$ is not the best point, and the boundary-effect mechanism's signature – a period approaching N always winning – is absent. Section 4.2 asks how much of this shape can actually be trusted statistically.

4.2 Statistical analysis

A sweep plot with 8 points invites two questions that a reader should not take on faith. First, is any individual period's apparent disadvantage relative to baseline real, or within the range that small-sample noise would produce? Second, is the period-to-period shape visible in Figure 2 a genuine effect, or just how the data happens to look at this particular scale? We address both questions directly, exploiting the paired-seed design of Section 3.4 throughout.

Periodic vs. baseline. For each of the 8 periods, we run a paired t-test (seed-matched) against the baseline, and apply Holm–Bonferroni correction (Holm, 1979) across the resulting family of 8 comparisons to control the family-wise error rate. Table 3 reports the results, together with Cohen's d_z (the paired effect size: mean of the paired differences divided by their standard deviation).

No individual period's disadvantage survives correction at $\alpha = 0.05$. Even the best uncorrected case, $P = 45$ with raw $p = 0.033$, is corrected to $p = 0.27$. At the same time, all 8 comparisons point in the same direction: periodic is always worse, with uniformly large paired effect sizes (d_z from 1.17 to 3.09). This pattern is more consistent with a real but small effect, one that 3 seeds and an 8-way correction simply lack the power to confirm individually, than with there being no effect at all. We report it accordingly: probably real, but not formally established at this sample

Table 3: Periodic vs. baseline, paired t-test with Holm–Bonferroni correction across the 8 comparisons.

P	Mean diff.	t	p (raw)	p (Holm)	Cohen's d_z
2	+0.0230	2.44	0.1348	0.4345	1.41
3	+0.0162	2.03	0.1791	0.4345	1.17
6	+0.0200	3.00	0.0954	0.4345	1.73
9	+0.0214	3.58	0.0700	0.4203	2.07
16	+0.0237	3.17	0.0869	0.4345	1.83
45	+0.0404	5.35	0.0332	0.2656	3.09
128	+0.0548	4.44	0.0471	0.3296	2.57
1024	+0.0414	3.13	0.0888	0.4345	1.81

size.

Is the period-to-period shape real? We next ask whether the 8 periodic configurations differ meaningfully *from one another*. We use two omnibus tests, both treating period as a within-subject factor and seed as the subject: a repeated-measures ANOVA, and a Friedman test as a non-parametric cross-check, appropriate given only 3 observations per cell. Both reject the null of no period effect decisively: $F(7, 14) = 21.62$, $p < 10^{-5}$ for the RM-ANOVA, and $\chi^2 = 19.0$, $p = 0.0082$ for the Friedman test. Each test's error term is the seed-by-period residual variance, so this is a direct comparison of the period-to-period spread against exactly the noise that varying the random seed produces. The answer to "is this shape just an artifact of the plot's scale" is therefore no: the spread in Figure 2 is larger than seed noise alone would produce, at high confidence.

This does not, however, mean that every visually apparent feature of the curve is individually confirmed. We computed all $\binom{8}{2} = 28$ pairwise period-vs-period comparisons, and Holm-corrected across that full family. This family includes the comparison between the single best period ($P = 3$) and the single worst ($P = 128$), which is the largest gap on the grid. Naively testing only that pre-selected extreme pair, uncorrected, gives $p = 0.0155$. But that pair was selected *by looking at the data*, as the lowest and highest of 8 means, so an uncorrected test on it is invalid. This is the same "testing the winner" problem that motivates using a corrected procedure such as Tukey's HSD, rather than an ordinary t-test, when comparing the extremes of several groups. Correcting properly, across the full 28-pair family, the best-vs-worst comparison is not significant ($p_{\text{Holm}} = 0.34$).

Taken together, these results support two separate claims at two different levels of confidence. The omnibus test establishes that real, non-noise structure exists across the 8 periods; this is a result that pools all the data into a single decision. No specific pairwise comparison between periods, including the largest one, is individually confirmed once properly corrected for having been selected from the data; this is a much narrower claim, with far less statistical power at $n = 3$. This is a standard and well-documented statistical pattern, not a contradiction: an omnibus test is routinely more powerful than its own post-hoc pairwise comparisons. It does mean, though, that the specific shape described at the end of Section 4.1 – low periods clustering, $P = 45/128$ worst, partial recovery at $P = 1{,}024$ – should be read as a visually suggestive, probably-real pattern, rather than as a set of individually established facts. More seeds would be needed to say with confidence which specific periods differ from which.

4.3 Training dynamics

The comparisons above are all evaluated at a single point: the best-validation-loss checkpoint of a finished run. We separately examined how the periodic-vs-baseline gap evolves *during* training, using the validation loss tracked every 100 steps (Section 3.4). Figure 3 shows all 10 configurations' validation loss against training step. Figure 4 isolates the periodic configurations and the baseline on their own scale, since RoPE's much larger gap would otherwise compress the more subtle structure we are interested in.

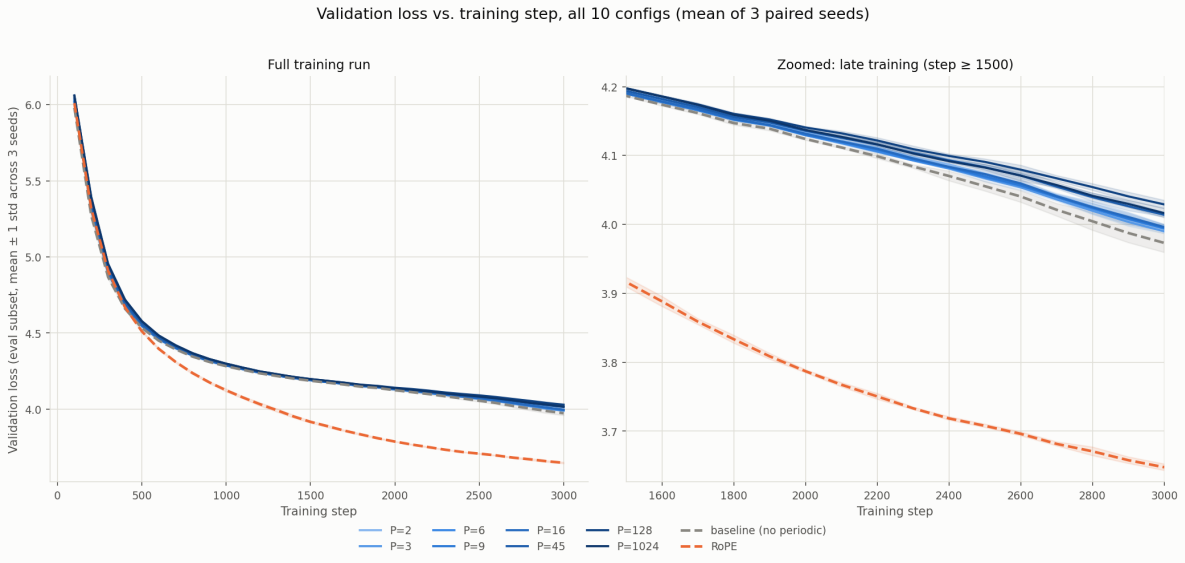


Figure 3: Validation loss vs. training step, all 10 configurations (mean of 3 paired seeds). Right: zoomed to the back half of training.

No training instability is visible in either plot. Every curve is smooth and monotonically decreasing, consistent with the complete absence of divergence or loss-spike events across all 30 runs (Section 4). A more specific pattern is visible instead: the periodic-vs-baseline gap is not constant over the course of training, but *U-shaped*. We measured this by taking the mean periodic configuration's validation loss minus the baseline's, at each tracked step. The gap starts at 0.055 at step 100, a value dominated by early optimization noise. It then shrinks steadily through the first ∼1,500 steps, roughly 2–3 of the run's 7–8 epochs, to a minimum of 0.006, essentially indistinguishable from baseline. From there it reopens, growing back to 0.030 by step 2,900, close to where the final-checkpoint comparison in Section 4.1 finds it. The spread *among* the 8 periodic configurations shows the identical shape: 0.047 at step 100, down to 0.008 at step 1,500, back up to 0.037 by step 2,900.

This is, to our knowledge, a new and unexplained observation: the periodic-vs-baseline disadvantage is specifically a late-training phenomenon. It does not look like a from-the-start optimization-speed penalty. The periodic and baseline models are nearly indistinguishable in the middle of training, and only diverge again in the final epochs. We return to what might explain this in Section 5. The obvious direct follow-up is examining the train/validation loss gap per configuration, to check whether periodic configurations show disproportionate late-epoch overfitting; we flag this as future work (Section 8).

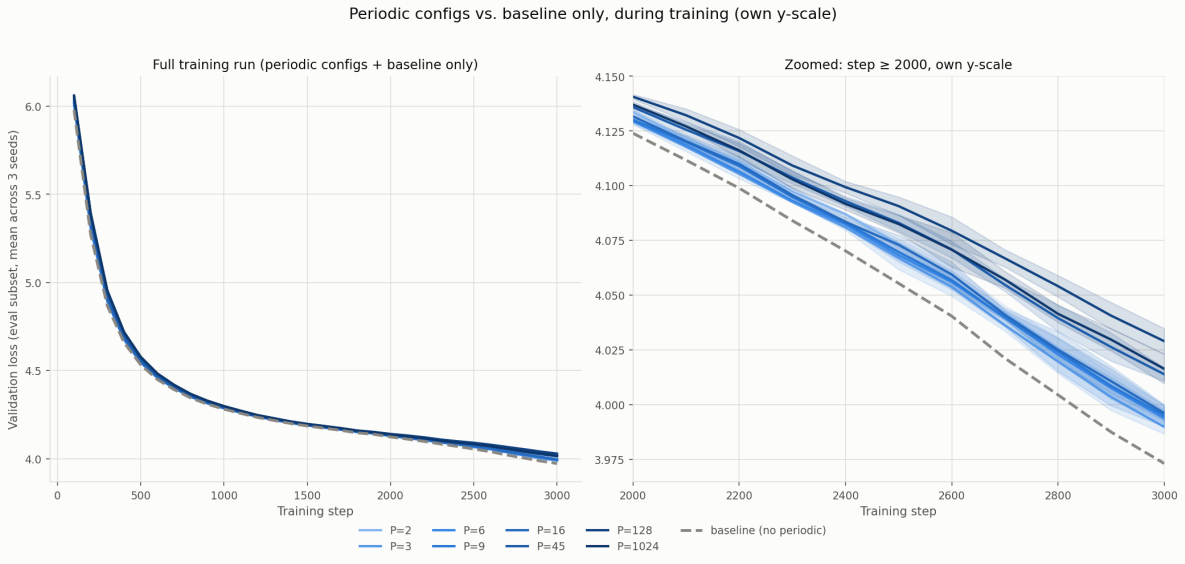


Figure 4: Periodic configurations and baseline only, on their own y-scale. Right: zoomed to step $\geq$ 2,000.

5 Discussion

This section is explicitly speculative. None of the mechanisms proposed here were tested directly; we label them as candidate explanations for the patterns in Section 4, worth stating plainly rather than leaving the results unexplained, but not to be read as established findings.

5.1 Why might the periodic component underperform baseline?

A priori, there is no reason the periodic component had to hurt. It could instead have supplied a useful inductive bias and sped up training, since nothing about adding an extra, cheaply-computed positional feature is obviously harmful. The fact that every periodic configuration underperforms baseline (Section 4.1) shows that if such a beneficial bias exists, it is outweighed by other effects. We consider three candidate mechanisms below, ordered from most to least plausible.

1. **A rigid periodic prior fighting soft, irregular data.** The periodic component imposes an exact-period assumption on the representation, regardless of whether the underlying text has anything genuinely periodic at that scale. Real clause and sentence lengths vary; they do not repeat on a fixed schedule. We consider this the most plausible of the three mechanisms. It predicts not merely that the periodic signal is *redundant*, which a model could simply learn to ignore at no cost, but that it is actively *misleading*. It supplies a specific wrong answer to the question "where am I in the period-P cycle," a question the data does not have a stable answer to, and the model may need to spend effort unlearning that wrong answer rather than merely discounting it.

2. **Integration cost under a fixed training budget.** Concatenation injection adds a larger learned projection than the baseline requires. The optimizer must learn to use this projection, or learn to down-weight it, within a fixed, short step budget (Section 3.4). Suppose the positional information it carries genuinely has nothing useful to add over the base encoding.

Even so, the model still has to spend some of its limited gradient steps discovering that fact; it cannot simply start from a representation that already excludes the redundant signal.

3. **A weaker injection channel than attention-native mechanisms.** RoPE (Section 3.1) acts multiplicatively, directly inside the attention computation. The periodic component here, by contrast, is concatenated into the input representation, and must propagate through several transformer layers before it can influence attention at all. This difference does not by itself explain a net-negative result: a weak channel for a genuinely useful signal should produce no benefit, not actual harm. It may, however, compound mechanisms 1 and 2, by making whatever useful signal exists harder to extract.

We read mechanism 1 as the leading explanation, with the other two as contributing factors that compound it, rather than treating the three as co-equal. A purely redundant signal (mechanisms 2 and 3 alone) predicts a model that eventually learns to ignore the extra input at little lasting cost. The training-dynamics result in Section 4.3 – a gap that nearly closes mid-training before reopening – points elsewhere: it is more consistent with something that goes wrong specifically in the later epochs, which fits a story about the periodic prior becoming actively misleading once the model has had enough passes over the data to start relying on it, than with a flat integration cost present from the start.

5.2 Why might different periods behave differently?

A second, independently speculative story concerns the shape within the periodic family itself (Section 4.1). One candidate account is that the *cost* of carrying the redundant periodic signal is not uniform across periods. Small periods ($P \leq 16$) oscillate fast enough within the window that they may be quickly and cleanly learned as uninformative. This is consistent with their tight clustering and low cross-seed variance in Table 2. Mid-range periods ($P = 45, 128$) sit roughly at the scale of real but irregular linguistic units, such as clause or sentence length in syllable-tokens. A signal that oscillates at a plausible-looking but unreliable timescale may be harder for the optimizer to cleanly discount than one that is obviously too fast to carry information. This would be consistent with $P = 128$ being the single worst point in the sweep. $P = 1{,}024 = N$, meanwhile, completes only one cycle across the entire context window. Within any training sequence, this makes it look like a smooth, slowly varying signal, rather than a genuinely periodic one, closer in character to the base encoding's own lowest-frequency components. It may therefore be "safely redundant" rather than actively confusing, which would be consistent with the partial recovery observed there. We note, without asserting any causal link, that this would be a near-mirror of the companion paper's own surprising finding, where a geometric property of the encoding correlated with trained performance in the *opposite* direction from what its design predicted. We flag this as a pattern worth watching for in follow-up work on this family of encodings, not as evidence in itself.

We emphasize again, per Section 4.2, that the specific shape this subsection discusses is statistically supported only in aggregate (the omnibus test), not pointwise – the speculation here concerns a pattern that is probably real but not yet individually confirmed period by period.

6 Limitations

Sample size. Three seeds per configuration is sufficient to establish an omnibus period effect, and to give consistent directional evidence for the periodic-vs-baseline comparison. It is not enough,

however, to individually confirm any single pairwise comparison, after correcting for the 8- or 28-way comparison families involved (Section 4.2). More seeds are the single most direct way to firm up every quantitative claim in this paper.

The tokenizer's known weaknesses (Section 2) bear on interpretation throughout. CMUdict reflects General American pronunciation, while TED speakers are international. Canonical dictionary syllable counts are not the same as syllables as realized in connected speech. And cut-placement, as opposed to count, is validated only against other heuristic tools, rather than against independent ground truth.

Corpus. TED-talk transcripts are edited subtitles of prepared, rehearsed talks, not a transcription of spontaneous speech. Whatever rhythmic structure syllable-tokenized text from this corpus does or does not exhibit may therefore not generalize to less formal registers of English.

Scale. The architecture, with 2.87M parameters and 3,000 training steps, is roughly one order of magnitude smaller than even modest modern language models. It was chosen to fit a 48-hour laptop compute budget. We cannot rule out that a larger model, trained for longer, would show a different result, including an opposite-signed one; the negative finding here is specific to this scale.

Mechanism, not just outcome. Section 5 offers a leading explanation for why this paper's period-vs-loss curve differs qualitatively from the companion paper's purely monotonic one: the presence of a base positional encoding here, which was absent there. This explanation is consistent with the data, and was anticipated during this project's design, before the data existed. It has not, however, been isolated by a direct ablation, training with and without the base encoding at matched P. It should be read as the leading candidate explanation, not as a demonstrated mechanism.

7 Related Work

Nabijou (2026) is the direct predecessor to this paper, and is discussed throughout; we do not repeat that discussion here. Beyond it, we found no prior work running this specific experiment: adding a periodic component to a standard positional encoding, and sweeping its period against training efficiency, in a from-scratch, syllable-tokenized GPT. As with the companion paper, then, this appears to be a novel combination rather than a replication of an established method. The closest relevant precedents are carried over from the companion paper's own survey, where they remain directly applicable:

Ghosh et al. (2024) ablate positional encoding in pretrained monolingual BERT models, across 22–23 languages and five downstream tasks, finding that PE importance diminishes with increasing morphological complexity. This remains the closest ML precedent for the idea that positional encoding matters differently depending on linguistic structure. It tests binary nullification of a standard, non-periodic PE on fine-tuned downstream tasks, though, rather than a period sweep on a from-scratch pretraining objective.

Stojanovic (2013) computes structural-complexity measures from written-text transcriptions across 21 languages, and finds that several proposed "rhythm" metrics correlate strongly with phonotactic complexity. This is direct supporting evidence for treating syllable structure, rather than raw character or timing counts, as the more principled unit for this family of hypotheses – precisely the design choice this paper makes (Section 2).

A broader literature search turned up concurrent work on syllable-aware tokenization specifically, independent of this paper's positional-encoding question. Atuhurra et al. (2024) introduce a syllable tokenizer for Swahili, and show that training GPT-2 from scratch on syllable-tokenized text produces more linguistically coherent embeddings than standard subword baselines; this echoes our own motivation that syllable-level units are a better fit than characters or BPE pieces for

languages with transparent syllable structure. Liao and Shi (2026) make the general case more directly, showing that standard subword tokenization systematically misaligns with syllable and rhyme boundaries across languages, and proposing a metric for that misalignment together with a lightweight fine-tuning method that retrofits phonological awareness onto an already-tokenized model. Our hybrid tokenizer takes the opposite approach: rather than retrofitting alignment after the fact, it builds the alignment in from the start, at the cost of being English-specific rather than a general-purpose fix. Nguyen et al. (2026) go a step further still, factorizing each syllable into its phonological onset, rime, and tone components and achieving extremely compact vocabularies (112 entries for Chinese, 256 for Vietnamese) while matching or exceeding character- and subword-level baselines; this is a finer-grained decomposition than the whole-syllable pieces used here, and a natural direction for extending this tokenizer in future work (Section 8). None of these three papers touches positional encoding, so none bears directly on this paper's headline experiment. Together, though, they suggest syllable-aware tokenization is an active and growing area, with the hybrid approach of Section 2 – anchoring counts to dictionary ground truth and falling back to heuristics only for out-of-vocabulary words – occupying a middle point between purely heuristic syllabifiers and fully learned phonological factorizations.

Su et al. (2021) introduce rotary position embedding, used here purely as a positive control (Section 3.1); its substantial empirical advantage over additive and concatenative positional encodings is well established and is not itself tested by this paper.

8 Conclusion and Future Work

Adding a periodic positional-encoding component on top of a standard base encoding does not improve GPT pretraining efficiency on English syllable-tokenized text, at the scale tested here. Every period swept performs worse than a no-periodic-component baseline, consistently in direction, though not individually significant at 3 seeds per configuration. Unlike the companion paper, this is not simply a boundary-effect artifact. The period-to-period structure is statistically real in aggregate, the worst-performing period is not the one closest to the context length, and a RoPE control confirms that the experimental setup can detect a genuine positional-encoding effect when one exists. A training-dynamics anomaly adds a further wrinkle: the periodic-vs-baseline gap very nearly closes mid-training, before reopening. This suggests the disadvantage is not a constant training-speed penalty. It behaves more like a late-training-specific effect, one not yet explained.

Independently of the periodic-encoding result, the hybrid syllable tokenizer of Section 2 is offered as a reusable artifact in its own right: a validated, well-behaved syllable-level unit for English that any project needing one – not only one about positional encoding – should be able to pick up directly.

Beyond collecting more seeds to resolve the statistical questions left open in Section 4.2, several concrete follow-ups would help adjudicate between the speculative mechanisms of Section 5:

- An ablation training with and without the base positional encoding at matched periods, to directly test whether its presence is what changes the shape of the period-vs-loss curve relative to the companion paper. The current inference is unablated.
- Tracking the train/validation loss gap per configuration specifically in the later training epochs, to test directly whether periodic configurations show disproportionate late-epoch overfitting, which the training-dynamics result (Section 4.3) suggests but does not establish.
- Inspecting the learned concatenation-projection weights, or probing intermediate activations,

to check whether the model down-weights the periodic input faster for small periods than for mid-range ones – a direct, cheap test of the "cheap vs. expensive to ignore" story in Section 5.

- A positive control on synthetic data with genuine, exact periodicity, to confirm the architecture *can* exploit real periodic structure when it is actually present – which would help establish that a negative result on natural text reflects a data/prior mismatch specifically, rather than the periodic component being unusable in this architecture at all.
- Measuring real structural periodicity directly in the syllable-token stream (e.g. clause- or sentence-length distributions), to test empirically whether the mid-range periods that perform worst really do coincide with genuine-but-irregular linguistic scales. Section 5 currently relies on that story by inference alone.
- A more attention-native (RoPE-like) injection of the same periodic idea, to separate the weak-injection-channel mechanism from the rigid-prior mechanism in Section 5: if a stronger channel still underperforms baseline, that would further implicate the rigid-prior explanation; if it recovers some benefit, that would implicate the channel instead.

Acknowledgements

Beyond the initial project design, this project's tokenizer, code, analysis, and this write-up were developed with the assistance of Claude (Anthropic). Prompt history is available on request.

References

Atuhurra, J., Shindo, H., Kamigaito, H., and Watanabe, T. (2024). Introducing syllable tokenization for low-resource languages: A case study with Swahili. *arXiv preprint arXiv:2406.15358*.

Cettolo, M., Girardi, C., and Federico, M. (2012). WIT^3: Web Inventory of Transcribed and Translated Talks. In *Proceedings of the 16th Conference of the European Association for Machine Translation (EAMT)*.

Carnegie Mellon University (1998). The CMU Pronouncing Dictionary. http://www.speech.cs.cmu.edu/cgi-bin/cmudict.

Ghosh, P., Vashishth, S., Dabre, R., and Bhattacharyya, P. (2024). A morphology-based investigation of positional encodings. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Liao, D. and Shi, F. (2026). How tokenization limits phonological knowledge representation in language models and how to improve them. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Nabijou, N. (2026). Language Cadence: A Null Result. Unpublished manuscript.

Nguyen, N. H., Huynh, T. B., Dinh-Le, B.-A., Hoang, P. G., Nguyen, D. T., Nguyen, K. V., and Nguyen, N. L.-T. (2026). Beyond atomic tokens: Factorizing syllables for language model pretraining. *arXiv preprint arXiv:2609.21362*.

Silva, L. and contributors (2013–). Pyphen: pure Python hyphenation. https://pyphen.org.

Stojanovic, D. (2013). *Cross-Linguistic Comparison of Rhythmic and Phonotactic Similarity*. PhD thesis, University of Hawai'i at Mānoa.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. (NLTK toolkit, https://www.nltk.org).